Salience Decay and TTL Eviction, Worked Out by Hand

How a memory's value bleeds away on a half-life, when it crosses the eviction floor, and how many memories that leaves resident in the store — every number counted by hand.

What this document does. It takes eviction policy 02 from Module 4.4 — *salience decay* — and works it on one episodic memory of the Customer-Support Resolver. We give the memory a starting value, a decay half-life, and an eviction *threshold*; derive the time-to-eviction in closed form; watch an *access* reinforce the memory and reset its clock; and finally use Little's law to turn that per-memory lifetime into the total number of memories left resident in the store at steady state. The module says "each recall bumps salience; non-recalled entries decay; below a floor → archive or drop" — we make that one sentence fully numerical. Every figure is reproduced by the small Python program in Appendix A.

Where this sits. This is **Topic 2 of Module 4.4** (Long-Term Memory Architectures) — the math behind eviction strategy 02, "Salience decay," and the `expires_at` TTL column of the `episodic_events` schema. Its companions: Module 4.4 Topic 1 (recency-weighted recall — the *same* exponential, used to rank instead of to forget) and the autonomy-cost reasoning of Module 0.0, where Little's law first turned a rate and a lifetime into a resident count.

Concepts covered, in order: the salience-decay law $v(t) = v_0 e^{-\lambda t}$ · the half-life relation $\lambda = \ln 2 / t_{1/2}$ · the eviction threshold θ · solving $v(t) = \theta$ for the closed-form TTL $= \ln(v_0/\theta)/\lambda$ · access-reinforcement resetting the clock · Little's law resident = rate × TTL · the storage-versus-recall dial.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one memory, decaying toward the floor

Unbounded memory is a leak. The module is blunt about it: "old, stale, contradictory entries pollute the top-k." Salience decay is the policy that fixes this automatically. Every memory carries a *salience* value $v \in [0, 1]$ (the schema defaults it to 0.5). That value drains over time unless the memory earns its keep by being recalled. When it sinks below an eviction *floor* θ, the memory is archived or dropped. No human, no nightly rule-list — just a number falling past a line.

We follow one episodic event: "user asked about `INV-1102`." Fresh, it matters; left untouched for weeks, it is noise. We give it round, hand-followable constants.

Quantity	Symbol	Value here
Initial salience (fresh write)	v_0	1.0
Eviction threshold (the floor)	θ	0.1
Salience half-life	$t_{1/2}$	14 days
Decay rate	$\lambda = \ln 2 / t_{1/2}$	0.0495 /day
New memories arriving per user	rate	100 /day

Notation. $v(t)$ is the memory's salience t days after its last reinforcement. TTL (time-to-live) is the age at which $v(t)$ first reaches the floor θ. We carry four decimals on values and one on days.

Intuition

Salience is a memory's *will to live*. It starts high on the day the memory is written, then leaks away on a half-life like a charge bleeding off a capacitor. Recalling the memory is the only way to top it back up. So the store quietly performs the thing human memory does: what you use stays sharp; what you ignore fades until it is gone. TTL is simply the moment the fade crosses the line where keeping the memory costs more than it is worth.

2 Step 1 — the decay law and its half-life

A memory's salience decays exponentially from its starting value:

$$v(t) \;=\; v_0\, e^{-\lambda t}, \qquad \lambda > 0,\; t \geq 0. \tag{1}$$

At $t = 0$, $v = v_0$ (full salience on write). As t grows, v slides toward 0 but never quite reaches it — which is exactly why we need a *threshold* to cut it off. As in the recency kernel of Topic 1, we set the rate from a half-life. Solving $v(t_{1/2}) = \frac{1}{2}v_0$:

$$e^{-\lambda t_{1/2}} = \tfrac{1}{2} \implies \lambda = \frac{\ln 2}{t_{1/2}} = \frac{0.6931}{14} = 0.0495 \text{ per day}. \tag{2}$$

Mini-example — this operation in isolation

The half-life makes the early values fall on clean fractions. $v(0) = 1.0$; $v(14) = \frac{1}{2} = 0.5000$ (one half-life); $v(28) = \frac{1}{4} = 0.2500$ (two half-lives). And $v(7) = e^{-0.0495 \cdot 7} = e^{-0.3466} = 0.7071$ — that is $1/\sqrt{2}$, the value at exactly half a half-life. So in the first week the memory keeps 71% of its salience; by two weeks, half; by four weeks, a quarter. The floor $\theta = 0.1$ sits below all of these — the memory has not been evicted yet at any of them.

3 Step 2 — solving for the time-to-eviction (TTL)

Eviction happens the instant salience touches the floor: $v(t) = \theta$. Set Equation (1) equal to θ and solve for t:

$$\begin{aligned} v_0\, e^{-\lambda t} &= \theta \\ e^{-\lambda t} &= \frac{\theta}{v_0} \\ -\lambda t &= \ln\frac{\theta}{v_0} = -\ln\frac{v_0}{\theta} \end{aligned}$$

$$\boxed{\text{TTL} \;=\; \frac{\ln(v_0/\theta)}{\lambda}} \tag{3}$$

Plug in $v_0 = 1.0$, $\theta = 0.1$, $\lambda = 0.0495$:

$$\text{TTL} = \frac{\ln(1.0/0.1)}{0.0495} = \frac{\ln 10}{0.0495} = \frac{2.3026}{0.0495} = 46.5 \text{ days.}$$

So this memory, written and never touched again, survives **46.5 days** before salience decay evicts it.

Mini-example — this operation in isolation

Verify by substituting TTL back into the decay law: $v(46.5) = 1.0 \cdot e^{-0.0495 \cdot 46.5} = e^{-2.3026} = 0.1000 = \theta$. ✓ The memory's value lands exactly on the floor at $t = 46.5$, as the closed form promised. Notice the TTL depends on v_0 and θ only through their *ratio*: doubling both leaves TTL unchanged, because $\ln(v_0/\theta)$ is unchanged. What sets the lifetime is how many *factors of ten* (here exactly one, since $v_0/\theta = 10$) the value must fall.

4 Step 3 — access reinforcement resets the clock

The module's rule is "each recall bumps salience." A memory that gets used should not be evicted on the same schedule as one that is ignored. Model an access at day 20 that *resets* salience to the full $v_0 = 1.0$ (the simplest bump — a reinforced memory is as good as new). Just before the access:

$$v(20) = 1.0 \cdot e^{-0.0495 \cdot 20} = e^{-0.9902} = 0.3715.$$

The memory was on its way down — 0.3715, heading for the floor at day 46.5. The access snaps it back to 1.0, and the decay clock restarts from day 20. Its new eviction time is

$$20 + \text{TTL} = 20 + 46.5 = 66.5 \text{ days.}$$

Scenario	salience at day 20	evicted at	lifetime
Never accessed	0.3715 (still falling)	day 46.5	46.5 d
Accessed at day 20	1.0 (reset)	day 66.5	66.5 d

One touch on day 20 bought the memory another full TTL — it now lives 66.5 days instead of 46.5. A memory recalled *repeatedly* effectively never dies: each access pushes the eviction line forward by up to a full TTL. That is the policy keeping the *useful* memories and dropping the noise, exactly as the module describes.

Watch out

Reinforcement is what makes salience decay smarter than a flat age-based TTL. A pure "delete everything older than 30 days" rule would evict a memory the user references every week. Salience decay does not — the weekly access keeps resetting it. But the same mechanism can *trap* a stale-but-frequently-recalled memory in the store forever. If a wrong fact keeps getting surfaced (and thus bumped), decay alone will never evict it; that is why the module stacks *contradiction resolution* and *dedup* on top. Decay handles the quiet noise; it does not handle confidently-wrong memories.

5 Step 4 — from lifetime to store size (Little's law)

A per-memory TTL answers "how long does one memory live?" The operational question is "how many memories are resident in the store at once?" Little's law connects them. For any system in steady state,

$$\underbrace{L}_{\text{items resident}} = \underbrace{\lambda_{\text{arr}}}_{\text{arrival rate}} \times \underbrace{W}_{\text{mean time in system}} . \tag{4}$$

Here arrivals are *new memories per day* and the mean time-in-system is the mean lifetime — the TTL. With 100 new memories per user per day and a 46.5-day TTL:

$$L \;=\; \text{rate} \times \text{TTL} \;=\; 100\ \tfrac{\text{items}}{\text{day}} \times 46.5\ \text{days} \;=\; 4{,}650\ \text{items resident.}$$

So a single user's episodic store settles at about **4,650 memories** — not because nothing is ever deleted, but because at steady state new writes exactly balance evictions. The store stops growing; it holds a rolling window one TTL deep.

> **Intuition**
>
> This is the same accounting as a bathtub with the tap running and the drain open. The water level stops rising when inflow equals outflow — and that level is (inflow rate)×(how long each drop stays). Memories arrive at 100/day and each lingers 46.5 days, so 4,650 sit in the tub. Want a smaller store? Either slow the tap (write fewer memories) or widen the drain (shorter TTL — a steeper decay or a higher floor).

6 Step 5 — TTL is the storage-versus-recall dial

TTL is not a fixed cost; it is a knob, and turning it trades storage against recall quality. Shorten the half-life (steeper decay) and TTL falls: fewer items resident, cheaper storage — but more memories evicted before they are next needed, which means more *cache misses*, each forcing a recompute (re-extract the fact, re-embed, re-store) or, worse, an answer given without the memory at all. Lengthen the half-life and the opposite holds: better recall, but a larger, costlier store. Sweeping the half-life with the closed form $\text{TTL} = \ln(10)/\lambda$ and Little's law:

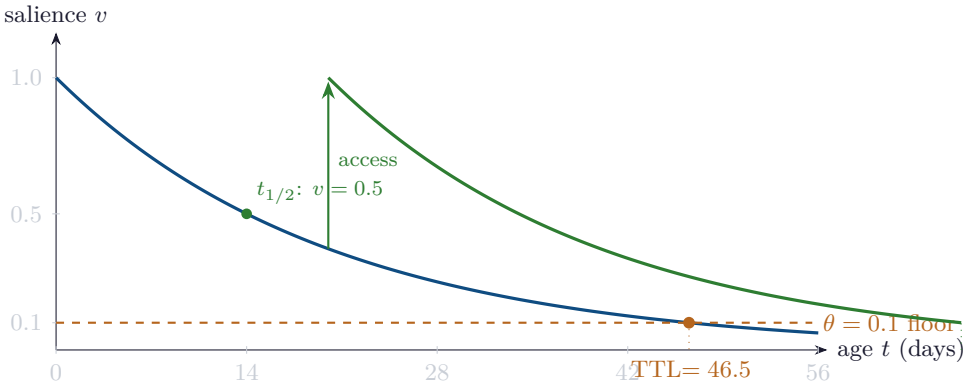
$t_{1/2}$	λ	**TTL** $= \ln 10/\lambda$	**resident** $= 100{\cdot}\text{TTL}$	**regime**
7 d	0.0990	23.3 d	2,325	lean store, more misses
14 d	0.0495	46.5 d	4,650	balanced (our setting)
30 d	0.0231	99.7 d	9,966	rich recall, heavier store

Halving the half-life from 14 to 7 days roughly halves both the TTL and the resident count (4,650 → 2,325) — because TTL is *linear* in $t_{1/2}$ (both $\text{TTL} = \ln 10 \cdot t_{1/2}/\ln 2$ and $L = \text{rate} \cdot \text{TTL}$ scale with $t_{1/2}$). Sample check, $t_{1/2} = 30$: $\lambda = \ln 2/30 = 0.02310$; $\text{TTL} = 2.3026/0.02310 = 99.7$ days; $L = 100 \times 99.7 = 9{,}966$. ✓

> **Intuition**
>
> There is no universally right TTL — only a right *point on the dial* for your domain and your budget. A throwaway chat assistant can run a short half-life and accept the misses; a

long-lived personal companion pays for a longer one because re-earning a forgotten fact is expensive and annoying. The decay law makes the trade-off a single tunable number rather than a pile of hand-written deletion rules — and, crucially, makes eviction *salience-aware* (used memories survive) rather than blindly age-based.

7 The decay and its floor, drawn



The blue curve is salience decaying from $v_0 = 1$ on its 14-day half-life; it crosses the orange floor $\theta = 0.1$ at TTL $= 46.5$ days, where the memory is evicted. The green arrow at day 20 is an access that resets salience to 1; the green curve is the new decay path, pushing eviction out to day 66.5. The whole picture is Equations (1) and (3): a half-life sets the slope, the floor sets the cut-off, and access restarts the clock.

8 Eviction cheat-sheet

Salience decay	$v(t) = v_0\, e^{-\lambda t}$
Half-life → rate	$\lambda = \ln 2 / t_{1/2}$
Eviction condition	evict when $v(t) < \theta$
Time-to-eviction	$\mathrm{TTL} = \ln(v_0/\theta)\,/\,\lambda$
Depends on v_0, θ via	their ratio only $(\ln(v_0/\theta))$
Access reinforcement	reset $v \to v_0$; eviction $\to t_{\text{access}} + \mathrm{TTL}$
Resident count (Little)	$L = \text{rate} \times \mathrm{TTL}$
Storage dial	shorter $t_{1/2} \Rightarrow$ smaller store, more misses

9 An honest word about these numbers

The constants here ($v_0 = 1.0$, $\theta = 0.1$, $t_{1/2} = 14$ days, 100 writes/day) are illustrative round numbers, chosen so the half-life values land on clean fractions and the TTL comes out as a tidy $\ln 10/\lambda = 46.5$ days. Real systems differ in every constant: salience may bump by a fixed increment rather than resetting to v_0, the arrival rate fluctuates by user and hour, and Little's law gives the *steady-state* resident count, so a brand-new or bursty store will not match it until

it settles. What is *not* illustrative is the structure: salience decays exponentially, the eviction time is $\ln(v_0/\theta)/\lambda$ in closed form, access reinforcement shifts that time forward by up to a full TTL, and the resident store size is rate × lifetime. Change the constants and the day-counts move; the shape — a tunable half-life dialing storage against recall, with eviction automatic and salience-aware — does not. That is why production memory stores decay rather than merely age out.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the decay values, the closed-form TTL, the reinforcement reset, and the storage table; change `v0`, `theta`, `half_life`, or `rate` to model your own store.

```python
# salience_ttl.py -- reproduces every number in "Salience Decay and TTL Eviction".
import math

V0 = 1.0 # initial salience on a fresh write
THETA = 0.1 # eviction floor: drop when v(t) < theta
HALF_LIFE = 14.0 # salience half-life, days
RATE = 100 # new memories per user per day
LAM = math.log(2) / HALF_LIFE # decay rate = ln2 / t_half

def value(t, v0=V0): # v(t) = v0 * exp(-lambda * t)
    return v0 * math.exp(-LAM * t)

def ttl(v0=V0, theta=THETA, lam=LAM): # solve v(t)=theta -> ln(v0/theta)/lambda
    return math.log(v0 / theta) / lam

print(f"lambda = ln2/{HALF_LIFE:.0f} = {LAM:.4f} /day")
T = ttl()
print(f"TTL = ln(v0/theta)/lambda = ln(10)/lambda = {T:.1f} days")
print(f"check v(TTL) = {value(T):.4f} (should be theta = {THETA})\n")

print("decay v(t):")
for t in (0, 7, 14, 28, T):
    print(f" t={t:5.1f} v={value(t):.4f}")

print("\naccess reinforcement at day 20:")
v20 = value(20)
print(f" v(20) before bump = {v20:.4f}")
print(f" reset to v0={V0}; new eviction at 20 + TTL = {20 + T:.1f} days")

print(f"\nLittle's law: resident = rate * TTL = {RATE} * {T:.1f}"
      f" = {RATE * T:.1f} items")

print("\nstorage-vs-recall dial (rate = 100/day):")
for hl in (7, 14, 30):
    lam = math.log(2) / hl
    t = math.log(V0 / THETA) / lam
    print(f" half_life={hl:2d}d TTL={t:5.1f}d resident={RATE * t:6.0f}")

# Expected output:
# lambda = ln2/14 = 0.0495 /day
# TTL = ln(v0/theta)/lambda = ln(10)/lambda = 46.5 days
# check v(TTL) = 0.1000 (should be theta = 0.1)
#
# decay v(t):
# t= 0.0 v=1.0000
# t= 7.0 v=0.7071
# t= 14.0 v=0.5000
```

```
# t= 28.0 v=0.2500
# t= 46.5 v=0.1000
#
# access reinforcement at day 20:
# v(20) before bump = 0.3715
# reset to v0=1.0; new eviction at 20 + TTL = 66.5 days
#
# Little's law: resident = rate * TTL = 100 * 46.5 = 4650.7 items
#
# storage-vs-recall dial (rate = 100/day):
# half_life= 7d TTL= 23.3d resident= 2325
# half_life=14d TTL= 46.5d resident= 4651
# half_life=30d TTL= 99.7d resident= 9966
```

— *end of document* —